Advanced Software Engineering VU [194.187]

Maria Christakis

TU Wien

https://mariachris.github.io

Bounded model checking for C: Theory

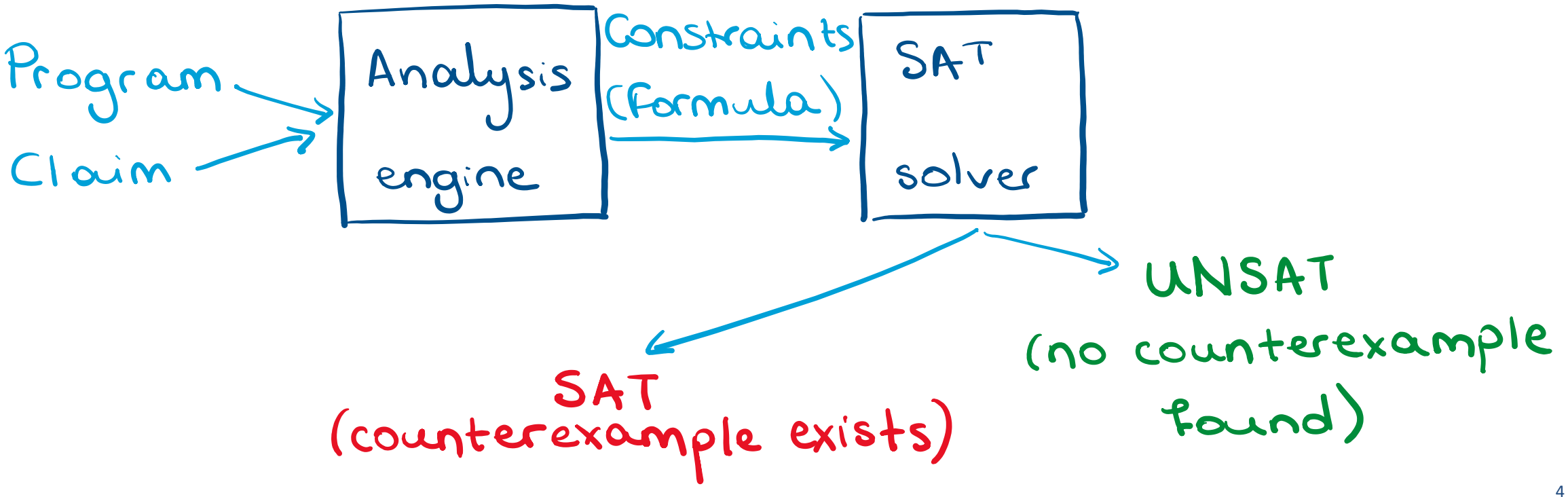
SAT solvers

- They are given a formula containing binary variables and logical relations such as $\wedge$ and $\vee$
  - For example, $(a \vee b) \wedge \neg c$, where a, b, and c can have values 0 (false) or 1 (true)
- They determine whether there exists an assignment to the formula variables such that the formula evaluates to true
- If they find such an assignment, they return SAT and the assignment
  - For example, a = 1, b = 0, and c = 0
- If they prove that no such assignment exists, they return UNSAT
  - For example, $a \wedge \neg a$

Bounded model checking

Main idea:

Given a program and a claim, use a SAT solver to find whether there exists an execution that violates the claim



Why use a SAT solver

- SAT solvers are very efficient (in solving an NP-hard problem)
- The analysis is completely automated
- The analysis is as good as the underlying SAT solver

Programs and claims

- Programs
  - Arbitrary C

- Claims
  - Array bound checks (that is, buffer overflow)
  - Division by zero
  - Pointer checks (that is, NULL pointer dereference)
  - Arithmetic overflow
  - User-provided assertions (for example, `assert i < j`)
  - …

Example: Correct claim

Program

```
int x;
int y = 8, z = 0; w = 0;
if (x) {
  z = y - 1;
} else {
  w = y + 1;
}
assert z == 7 || w == 9;
```

Constraints

$$
\begin{aligned}
& y = 8 \wedge \\
& z = (x\ ?\ y - 1 : 0) \wedge \\
& w = (x\ ?\ 0 : y + 1) \wedge \\
& z \neq 7 \wedge \\
& w \neq 9
\end{aligned}
$$

UNSAT

no counterexample

assertion always holds

Example: Incorrect claim

Program

```
int x;
int y = 8, z = 0; w = 0;
if (x) {
  z = y - 1;
} else {
  w = y + 1;
}
assert z == 5 || w == 9;
```

Constraints

$$
\begin{aligned}
& y = 8 \wedge \\
& z = (x\ ?\ y - 1 : 0) \wedge \\
& w = (x\ ?\ 0 : y + 1) \wedge \\
& z \neq 5 \wedge \\
& w \neq 9
\end{aligned}
$$

SAT

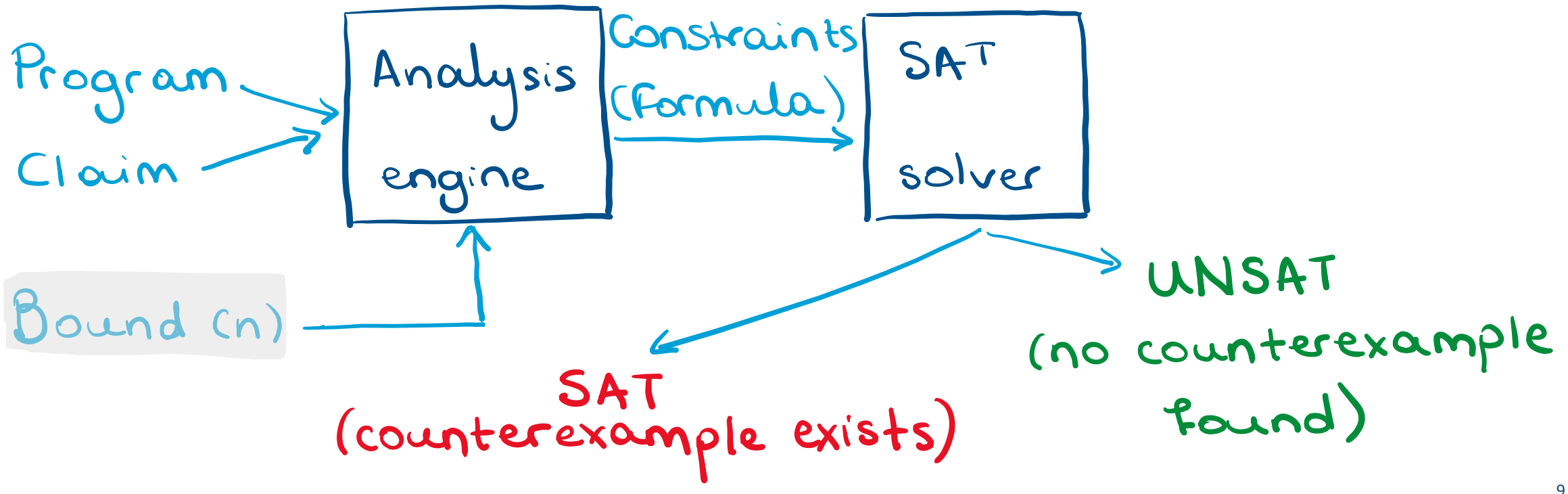
Counterexample found

$y=8, x=1, w=0, z=7$

What about loops

- SAT solvers can only explore finite-length executions

- Loops must be bounded, making the analysis potentially unsound



How does it work

1. Simplify control flow
2. Unroll all loops
3. Convert into Single Static Assignment (SSA) form
4. Convert into constraints
5. Bit-blast
6. Solve with a SAT solver
7. Convert SAT assignment into a counterexample

Control-flow simplifications

- All side effects are removed
  - For example, `j = i++` becomes `j = i; i = i + 1`
- Control flow is made explicit
  - `continue` and `break` are replaced by `goto`
- All loops are simplified into one form
  - `for` and `do … while` are replaced by `while`

Loop unrolling

- All loops are unrolled
  - Including backward `goto` jumps and recursive functions
- Different unrolling bounds can be used for different loops
- Special unrolling assertions or assumptions are used to handle executions that exceed the chosen loop-unrolling bound

Loop unrolling

```
void f(…) {
  …
  while (cond) {
    <Body>
  }
  <Remainder>
}
```

loop unrolling →

with an unrolling bound of 3

```
void f(…) {
  …
  if (cond) {
    <Body>
    if (cond) {
      <Body>
      if (cond) {
        <Body>
        while (cond) {
          <Body>
        }
      }
    }
  }
  <Remainder>
}
```

Loop unrolling: Unrolling assertion

```
void f(…) {
  …
  if (cond) {
    <Body>
    if (cond) {
      <Body>
      if (cond) {
        <Body>
        while (cond) {
          <Body>
        }
      }
    }
  }
  <Remainder>
}
```

The unrolling assertion replaces the while loop

It is violated if the loop does more iterations than the bound permits

```
void f(…) {
  …
  if (cond) {
    <Body>
    if (cond) {
      <Body>
      if (cond) {
        <Body>
        assert !cond;
      }
    }
  }
  <Remainder>
}
```

Loop unrolling: Unrolling assumption

```
void f(…) {
  …
  if (cond) {
    <Body>
    if (cond) {
      <Body>
      if (cond) {
        <Body>
        while (cond) {
          <Body>
        }
      }
    }
  }
  <Remainder>
}
```



```
void f(…) {
  …
  if (cond) {
    <Body>
    if (cond) {
      <Body>
      if (cond) {
        <Body>
        assume !cond;
      }
    }
  }
  <Remainder>
}
```

Converting into SSA form

- When a variable is assigned multiple times, we use a new variable for the right-hand side of each assignment

Program

```
x = x + y;
x = x * 2;
```

SSA Program

```
x1 = x0 + y0;
x2 = x1 * 2;
```

- If we do not convert into SSA form, our constraints will be incorrect
  - For example, $x = x + y \wedge x = x * 2$

Converting into SSA form: Conditionals

Program

```
if (v) {
  x = y;
} else {
  x = z;
}
w = x;
```

SSA Program

```
if (v0) {
  x0 = y0;
} else {
  x1 = z0;
}
w = ???;
```

Converting into SSA form: Conditionals

Program

```
if (v) {
  x = y;
} else {
  x = z;
}
w = x;
```

SSA Program

```
if (v0) {
  x0 = y0;
} else {
  x1 = z0;
}
x2 = (v0 ? x0 : x1);
w = x2;
```

Converting into constraints

- We convert the program P and the claims C into the formula $P \wedge \neg C$
  - Note that C is a conjunction of all claims

Program

```
x = x + y;
if (x != 1) {
  x = 2;
} else {
  x++;
}
assert x <= 3;
```

SSA Program

```
x1 = x0 + y0;
if (x1 != 1) {
  x2 = 2;
} else {
  x3 = x1 + 1;
}
x4 = ((x1 != 1) ? x2 : x3);
assert x4 <= 3;
```

Constraints

$$
\begin{aligned}
&x_1 = x_0 + y_0 \wedge \\
&x_2 = 2 \wedge \\
&x_3 = x_1 + 1 \wedge \\
&x_4 = (x_1 \mathrel{!}= 1) \;?\; x_2 : x_3 \wedge \\
&x_4 > 3
\end{aligned}
$$

Converting into constraints: Unrolling assertions

- We convert the program P and the claims C into the formula $P \wedge \neg C$ using unrolling assertions
  - Note that unrolling assertions are part of the claims
  - If the formula is SAT, then the claims do not hold or an unrolling assertion does not hold
  - If the formula is UNSAT, then the claims hold
  - Unrolling assertions make the analysis sound in the presence of loop unrolling
  - Use case: verification

Converting into constraints: Unrolling assumptions

- We convert the program P and the claims C into the formula $P \wedge \neg C$ using unrolling assumptions
  - Note that assumption conditions are not part of the claims; they are part of the program and should not be negated in the constraints
  - If the formula is SAT, then the claims do not hold
  - If the formula is UNSAT, then the claims hold or they were not checked due to the unrolling assumptions
  - Unrolling assumptions make the analysis unsound in the presence of loop unrolling
  - Use case: bug finding

Bit-blasting

- Bit-blasting converts operations on integers (in the form of bit vectors) into operations for binary variables
  - This is needed to provide constraints with binary variables to the SAT solver
- This tremendously blows up the formula size
- Assume the constraints use 5 integer variables (e.g., x_0, x_1, x_2, y_0, y_1) of bit-width 32, how many binary variables are there after bit-blasting?

Bit-blasting

- Bit-blasting converts operations on integers (in the form of bit vectors) into operations for binary variables
  - This is needed to provide constraints with binary variables to the SAT solver
- This tremendously blows up the formula size
- Assume the constraints use 5 integer variables (e.g., x_0, x_1, x_2, y_0, y_1) of bit-width 32, how many binary variables are there after bit-blasting?
  - 160!

How does it work

1. Simplify control flow
2. Unroll all loops
3. Convert into Single Static Assignment (SSA) form
4. Convert into constraints
5. Bit-blast
6. Solve with a SAT solver
7. Convert SAT assignment into a counterexample

CBMC

- Developed at CMU by Daniel Kroening et al.
- Available at: http://www.cprover.org/cbmc/
- Known to scale to programs with over 30K lines of code
- Was used to find previously unknown bugs in MS Windows device drivers

Bounded model checking for C: Exercises

Exercise 5: Bounded Model Checking

Consider the C function below and try to verify it by performing Bounded Model Checking. Use a loop unrolling bound of 2 and perform every step of Bounded Model Checking up until the Bit-Blasting step. Would a SAT solver invoked by the Bounded Model Checker finally output SAT or $UNSAT$? In case it would output SAT, give a satisfying assignment of the integer variables in the constraints constructed. In case it would output $UNSAT$, explain why such an assignment cannot exist. What can we learn from the output of the SAT solver about the safety of the program?

```
void sum(unsigned int n){
  unsigned int i = 0;
  unsigned int s = 0;
  if(n < 10000){
    while(++i < 5){
        s+= n;
    }
  }
  assert(s <= 5*n);
}
```

Exercise 5: Bounded model checking

- What happens if I use unrolling assumptions?

- What happens if I change `while (++i < 5)` to `while (++i < 3)`?